

THE CHAISE LONGUE THEOREM — LEAN CERTIFICATE v2

A machine-checked proof, in Lean 4 with Mathlib, of Main Theorem' for every degree: the algebraic core of Conjecture 1.2 of Degtyarev–Shimada for the Fermat varieties of every degree $m \geq 1$ in every even dimension

Rafael Amichis Luengo · Madrid, Spain · tretoef@gmail.com

Certificate v2 · 3 October 2026 · aligned with version 12 of the paper, and corrected after its cold reading, on 4 October 2026

Companion to *The Chaise Longue Theorem*, version 12. The numbers of sections and results below are those of version 12. They coincide with those of version 11, which was never released, for every result cited here except three in §8.5: Lemma 8.7' is new in version 12, and Lemma 8.8 and Corollary 8.9 are new statements under numbers that version 11 used for its route through an exterior algebra (paper, §1.8). Version 1 of this certificate (29 September 2026, DOI 10.5281/zenodo.23045371) covers the odd degrees and is unchanged; this version extends it to every degree and adds Theorem O.

0. Verdict

For **every** $m \geq 1$ and every $k \geq 0$, Lean 4 proves:

- the ring $R = \mathbb{Z}[t_1, \dots, t_{2k+1}]/(t_1^m - 1, \dots, t_{2k+1}^m - 1)$ modulo the ideal generated by the $(2k+1)!!$ elements ψ_J of Degtyarev–Shimada is a **free abelian group of rank $m^{2k+1} - Q_k(m)$** ;
- for **every field** F , $\dim_F F[G]/(\psi_J) = m^{2k+1} - Q_k(m)$, and $\dim_F (\psi_J) = Q_k(m)$.

Here $Q_k(m) = N! \cdot [x^N] I_0(2x)^{(m-1)/2}$ for odd m and $Q_k(m) = N! \cdot [x^N] \cosh(x) \cdot I_0(2x)^{(m-2)/2}$ for even m , with $N = 2k+2$ and $I_0(2x) = \sum_b x^{2b}/(b!)^2$ (paper §1.2). This is Main Theorem' of the paper (§1.2; Corollary 7.7 for odd m , Theorem 9.11 for even m), stated there for $m \geq 3$ and $k \geq 1$. It holds for every prime at once, the prime 2 and the primes dividing m included.

Lean 4 also proves **Theorem O with equality** (paper §1.5, §8.7 and Corollary 8.13): for every field F , every odd $r \geq 3$ and every $k \geq 0$, $\dim_F (D_{r,J} : J) \cdot F[y_1, \dots, y_{2k+1}]/(y_i^r) = N! \cdot [x^N] \cosh(x) \cdot I_0(2x)^{(r-1)/2}$, and the integral quotient is a free abelian group.

Every module that holds a theorem compiles on the author's Mac (229 of the 233 modules; the other four hold numeric checks only, §2 item 4 and §8). The proof contains no sorry, no admit, no native_decide and no added axiom. Every main theorem depends only on the three standard axioms of Lean and Mathlib: propext, Classical.choice and Quot.sound.

Main Theorem' is not the Main Theorem. The Main Theorem is Conjecture 1.2 itself: $H_{2k}(X; \mathbb{Z})/L(X)$ is torsion free. It follows from Main Theorem' by one topological result, [DS, Theorem 1.1(a)]: the torsion of $R/(\psi_J)$ equals the torsion of $H_{2k}(X; \mathbb{Z})/L(X)$. That result rests on Pham's description of the homology of Fermat varieties and on the intersection numbers of [DS, Theorem 2.2]. **It is not formalized; it is cited.** So:

Lean certifies the algebraic statement (Main Theorem') completely, for every degree. The Main Theorem follows from it by one cited topological result, the same one the paper cites.

Read cold. This certificate was read cold before release, on 4 October 2026, by a separate reader (an instance of Claude working alone from a written mission, with the certificate, the paper, the original of [DS], the pieces and the compiled project). Its verdict: «HOLDS AS FAR AS I CAN CHECK». It printed the final theorem and Theorem O from the compiled files and compared them with the paper and with [DS]; counted Γ directly from [DS, Definition 1.3] in Python and in Lean; re-elaborated five files of the chain from source; re-counted every number of this certificate; and, beyond its mission, re-submitted to the Lean kernel every one of the 4 669 project declarations on which the final theorem and Theorem O depend, with a negative control. It found no FATAL and no GAP, four ERRORS of numbers or wording and five points of presentation, all in this certificate and none in the proofs; all are corrected in this text (§9). Its report and code: [Rep], record/cold-readings/17_internal-reader_on-lean-certificate-v2/.

§1 states exactly what is proved. §2 states exactly what is not. §3–§8 describe how the result was obtained and how to check it. §9 lists what changed from version 1.

1. The certified statement

1.1 The theorem, verbatim

Printed by Lean with #check (log CERTIFICADO_PARES/check_pares_postclean.log, from checks/CheckPares.lean, run against the clean rebuild of §8):

```
@EvenAll.mainTheorem' : ∀ {m : ℕ},
  1 ≤ m →
  ∀ (k : ℕ),
    (Module.Free ℤ (RZ (2 * k + 1) m / IZ m k) ∧
     Module.finrank ℤ (RZ (2 * k + 1) m / IZ m k) = m ^ (2 * k + 1) - EvenAll.Qall m k) ∧
    ∀ (F : Type u_1) [inst : Field F],
      Module.finrank F (GA F (2 * k + 1) m / IK F m k) = m ^ (2 * k + 1) - EvenAll.Qall m k ∧
      Module.finrank F (Submodule.restrictScalars F (IK F m k)) = EvenAll.Qall m k
```

'EvenAll.mainTheorem'' depends on axioms: [propext, Classical.choice, Quot.sound]

The function $Qall$ is $Q_k(m)$ with the parity split of the paper:

```
def EvenAll.Qall : ℕ → ℕ → ℕ :=
fun m k => if Even m then EvenCount.QkEven k m else TheoremB.Qk k m
def EvenCount.QkEven : ℕ → ℕ → ℕ :=
fun k m =>
  ∑ c ∈ Finset.range (k + 2),
    ∑ b ∈ Fintype.piFinset fun x => Finset.range (2 * k + 3) with 2 * c + 2 * ∑ u, b u = 2 * k + 2,
      (2 * k + 2).factorial / ((2 * c).factorial * ∏ u, (b u).factorial ^ 2)
def TheoremB.Qk : ℕ → ℕ → ℕ :=
fun k q =>
  ∑ b ∈ Fintype.piFinset fun x => Finset.range (2 * k + 3) with 2 * ∑ u, b u = 2 * k + 2,
    (2 * k + 2).factorial / ∏ u, (b u).factorial ^ 2
```

For odd m the statement is that of version 1 (ColAssembly.mainTheorem', still present and unchanged; $Qall\ m\ k = TheoremB.Qk\ k\ m$ there). For even m it is EvenAll.B2 together with EvenAll.B1. The field part quanti-

fies over fields in every universe; CheckPares.lean restates it by hand for $F : \text{Type}$ and Lean accepts the restatement.

1.2 The definitions it uses, verbatim

Printed by #print in the same log. The ring, the ideal and the generators are those of version 1 (§1.2 there), which are unchanged; they are repeated here so that this certificate can be read alone.

```
def ColAssembly.idealZ : (d : ℕ) → ℕ → Ideal (MvPolynomial (Fin d) ℤ) :=
fun d m => Ideal.span (Set.range fun i => MvPolynomial.X i ^ m - 1)
@[reducible] def ColAssembly.RZ : ℕ → ℕ → Type :=
fun d m => MvPolynomial (Fin d) ℤ / idealZ d m
def ColAssembly.IZ : (m k : ℕ) → Ideal (RZ (2 * k + 1) m) :=
fun m k => Ideal.span (Set.range fun J => (Ideal.Quotient.mk (idealZ (2 * k + 1) m)) (psiP ℤ m J))
def ColUpper.psiP.{u_1} : (R : Type u_1) →
[inst : CommRing R] → ℕ → {k : ℕ} → BallotBound.Matching k → MvPolynomial (Fin (2 * k + 1)) R :=
fun R [CommRing R] m {k} J =>
(∏ a with a < ↑J a, (tP R k (↑J a) - 1)) * ∏ a with 0 < a ∧ a < ↑J a, ColSurv.phi m (tP R k a * tP R
k (↑J a))
def ColUpper.tP.{u_1} : (R : Type u_1) →
[inst : CommRing R] → (k : ℕ) → Fin (2 * k + 2) → MvPolynomial (Fin (2 * k + 1)) R :=
fun R [CommRing R] k i => Fin.cases 0 MvPolynomial.X i
def ColSurv.phi.{u_2} : {A : Type u_2} → [Semiring A] → ℕ → A → A :=
fun {A} [Semiring A] m u => ∑ s ∈ Finset.range m, u ^ s
def BallotBound.Matching : ℕ → Type :=
fun k => { J // ∀ (x : Fin (2 * k + 2)), J x ≠ x ∧ J (J x) = x }
def ColSplit.gaIdeal.{u_1} : (F : Type u_1) → [inst : Field F] → (d : ℕ) → ℕ → Ideal (MvPolynomial
(Fin d) F) :=
fun F [Field F] d m => Ideal.span (Set.range fun i => MvPolynomial.X i ^ m - 1)
@[reducible] def ColSplit.GA.{u_1} : (F : Type u_1) → [Field F] → ℕ → ℕ → Type u_1 :=
fun F [Field F] d m => MvPolynomial (Fin d) F / gaIdeal F d m
def ColUpper.IK.{u_1} : (K : Type u_1) → [inst : Field K] → (m k : ℕ) → Ideal (GA K (2 * k + 1) m) :=
fun K [Field K] m k => Ideal.span (Set.range (psiG K m))
def ColUpper.psiG.{u_1} : (K : Type u_1) →
[inst : Field K] → (m : ℕ) → {k : ℕ} → BallotBound.Matching k → GA K (2 * k + 1) m :=
fun K [Field K] m {k} J =>
(∏ a with a < ↑J a, (tU K m k (↑J a) - 1)) * ∏ a with 0 < a ∧ a < ↑J a, ColSurv.phi m (tU K m k a *
tU K m k (↑J a))
def ColUpper.tU.{u_1} : (K : Type u_1) → [inst : Field K] → (m k : ℕ) → Fin (2 * k + 2) → GA K (2 * k
+ 1) m :=
fun K [Field K] m k i => Fin.cases 0 (fun j => (Ideal.Quotient.mk (gaIdeal K (2 * k + 1) m))
(MvPolynomial.X j)) i
```

tU is the image of tP in the quotient ring: $t_0 = 0$ and t_j is the class of the variable X_j , as in the integral version tP. The #print of tU was added to CheckPares.lean after its first run, so it appears only in the post-clean log.

1.3 Faithfulness to the paper, line by line

The rows for $\mathcal{J}$, R , φ , τ_j , ψ_j , t_0 , the ideal, $F[G]$ and «free abelian group of rank r » are those of version 1 §1.3, unchanged. The new rows are the even count and the parity split.

Paper	Lean	Agreement
$Q_k(m) = N! \cdot [x^N] \cosh(x) \cdot I_0(2x)$ ($m-2)/2$ for even m (§1.2, §9.1)	$\text{QkEven } k \ m = \sum_{\{c, b\} : \mathbb{N}! / ((2c)! \cdot \prod_u (b_u!)^2) \text{ over } c + \sum_u b_u = k+1, b : \text{Fin } ((m-2)/2) \rightarrow \mathbb{N}}$	$\cosh(x) = \sum_c x^{2c} / (2c)!$ and $I_0(2x) = \sum_b x^{2b} / (b!)^2$, so the coefficient of x^N in $\cosh(x) \cdot \prod_{u=1}^{(m-2)/2} I_0(2x)$ times $N!$ is exactly this sum. Each quotient is a multinomial coefficient (parts $2c, b_u, b_u$), so the division in $\mathbb{N}$ is exact. The bounds $c \leq k+1$ and $b_u \leq N$ of the Finsets lose nothing.

Paper	Lean	Agreement
$Q_k(m) = \Gamma_{\mathcal{J}} $ for even m (Lemma 9.1)	<code>EvenCount.card_Gamma_even: (Gamma hp rod k).card = QkEven k m</code> over a field with $m \neq 0$ and m distinct m -th roots of unity	The set Γ of [DS, Definition 1.3]. In Lean, <code>ColUpper.Gamma</code> is the set of points where some ψ_j does not vanish, which is [DS, Definition 1.3] by the argument of [DS, Claim 4.3]; <code>card_Gamma_eq_card_pairs</code> then counts the tuples that split into inverse pairs. The cold reader counted Γ literally from Definition 1.3, in Python and in Lean, in 30 cells, and found Q_{all} equal in every one.
«the number of N -tuples in a set of r elements with an involution that has one fixed point, which split into pairs of opposite entries» (§1.5, Lemma 9.1)	<code>EvenCount.card_closedPointed: (closedPointed S (2k+2)).card = QkEven k (2h+2)</code> for every pointed set S with h classes	The paper's combinatorial reading.
$Q_k(m)$ with the parity split (§1.2)	<code>Qall m k = if Even m then QkEven k m else TheoremB.Qk k m</code>	Literal.
$m^{2k+1} - Q_k(m)$	<code>m (2k+1) - Qall m k</code> in $\mathbb{N}$	Never truncates: the field part proves <code>Qall m k = dim_F I_F ≤ dim_F F[G] = m^{2k+1}</code> .

Further checks in `CheckPares.lean`, by kernel computation (decide; the first three through `EvenCount.QkEven_values`, itself proved by decide), tie the definitions to their values: `QkEven 1 4 = 19`, `QkEven 2 4 = 141`, `QkEven 1 6 = 61`, `QkEven 0 4 = 3`, `QkEven 1 2 = 1`, `Qk 1 3 = 6`, `Qk 1 5 = 36`, `Qall 4 1 = 19`, `Qall 3 1 = 6`. And the final theorem is shown **not vacuous** at the Fermat quartic surface: Lean derives from `mainTheorem'` that $RZ \ 3 \ 4 / IZ \ 4 \ 1$ is free of rank $45 = 4^3 - 19$, and that its dimension is 45 over F_2 , over F_3 and over $\mathbb{Q}$ — the prime 2 divides the degree, the prime 3 does not, and $\mathbb{Q}$ has characteristic 0.

Generality. The paper states Main Theorem' for $m \geq 3$ and $k \geq 1$. Lean proves it for every $m \geq 1$ and every $k \geq 0$, and the field part for every field, not only F_p . The degenerate cases are consistent: at $m = 1$, $R = Z$ and the rank is 1; at $m = 2$, $Q_k(2) = 1$ (`Pow2.q_two`); at $k = 0$ it is the paper's trivial case.

1.4 Theorem O with equality (piece E15)

Printed verbatim by Lean (logs: `check_pares_postclean.log` in `CERTIFICADO_PARES/`, and `check_runE15.log` in `proyecto_lean/`):

```
OddEquality.D1 : ∀ (F : Type u_1) [inst : Field F] {h : ℕ},
  1 ≤ h →
    ∀ (k : ℕ),
      Module.finrank F (Submodule.restrictScalars F (TheoremB.DIdeal F (2 * h + 2) k)) =
        EvenCount.QkEven k (2 * h + 2) ∧
        Module.finrank F (Submodule.restrictScalars F (OddEquality.Mideal F (2 * h + 2) (2 * k + 2)))
=
  EvenCount.QkEven k (2 * h + 2)
@OddEquality.D2_DJ : ∀ {h : ℕ},
  1 ≤ h →
    ∀ (k : ℕ),
      Module.Free ℤ (FreeZ.CZ (2 * h + 2) (2 * k + 1) / OddEquality.DIdealZ (2 * h + 2) k) ∧
      Module.finrank ℤ (FreeZ.CZ (2 * h + 2) (2 * k + 1) / OddEquality.DIdealZ (2 * h + 2) k) =
        (2 * h + 1) ^ (2 * k + 1) - EvenCount.QkEven k (2 * h + 2)
@OddEquality.D2_M : ∀ {h : ℕ},
  1 ≤ h →
    ∀ (k : ℕ),
      Module.Free ℤ (FreeZ.CZ (2 * h + 2) (2 * k + 2) / OddEquality.MidealZ (2 * h + 2) (2 * k + 2)) ∧
      Module.finrank ℤ (FreeZ.CZ (2 * h + 2) (2 * k + 2) / OddEquality.MidealZ (2 * h + 2) (2 * k +
2)) =
```

```

(2 * h + 1) ^ (2 * k + 2) - EvenCount.QkEven k (2 * h + 2)
OddTheorem.theorem0 : ∀ (F : Type u_1) [inst : Field F] {h : ℕ},
  1 ≤ h →
    ∀ (k : ℕ),
      EvenCount.QkEven k (2 * h + 2) ≤ Module.finrank F (Submodule.restrictScalars F (TheoremB.DIdeal
F (2 * h + 2) k))

```

Here $\text{TheoremB.DIdeal } F (2h+2) k$ is the ideal $(D_{r,J} : J \in \mathcal{J})$ of $F[y_1, \dots, y_{2k+1}]/(y_i^r)$ with $r = 2h+1$ (the box ring $\text{Peel.C } F q m = F[y]/(y_i^{q-1})$ at $q = 2h+2$), $D_r(a, b) = \sum_{u < r} (-1)^u a^u b^{r-1-u}$, and $D_{r,J}$ the product of the $D_r(y_a, y_b)$ over the pairs of J that avoid 0 — the definition TheoremB.DIdeal of Theorem B, used here with an even q , so that the exponent $q - 1 = r$ of the box is odd; $\text{OddEquality.Mideal}$ is the companion ideal $\mathcal{M} = \sum_P D_P \cdot C_N$ of Lemma 8.12 over all perfect matchings P of $\text{Fin } N$. $\text{QkEven } k (2h+2)$ is the right-hand side $N! \cdot [x^N] \cosh(x) \cdot I_0(2x)^h$ of the paper (row 1 of §1.3). The integral quotients are taken literally by the ideals generated by the integer polynomials $D_J(\text{OddEquality.DIdealZ})$ and $D_P(\text{OddEquality.MidealZ})$.

The definitions behind Theorem O, printed verbatim by Lean (checks/CheckTheorem0.lean, log proyecto_lean/check_theorem0.log, written after the cold reading; exit code 0):

```

def TheoremB.DIdeal.{u_1} : (F : Type u_1) → [inst : Field F] → (q k : ℕ) → Ideal (Peel.C F q (2 * k + 1)) :=
fun F [Field F] q k => Ideal.span (Set.range (TheoremB.DJ F q))
def TheoremB.DJ.{u_1} : (F : Type u_1) →
  [inst : Field F] → (q : ℕ) → {k : ℕ} → BallotBound.Matching k → Peel.C F q (2 * k + 1) :=
fun F [Field F] q {k} J => [] a with a ≠ 0 ∧ ↑J a ≠ 0 ∧ a < ↑J a, Tight.D F q (TheoremB.idx a)
(TheoremB.idx (↑J a))
def TheoremB.idx : {k : ℕ} → Fin (2 * k + 2) → Fin (2 * k + 1) :=
fun {k} a => (↑a - 1, ...)
def Tight.D.{u_1} : (F : Type u_1) → [inst : Field F] → (q : ℕ) → {m : ℕ} → Fin m → Fin m → Peel.C F q m :=
fun F [Field F] q {m} a b => ∑ i ∈ Finset.range (q - 1), (-1) ^ i * Tight.y F q a ^ i * Tight.y F q b ^ (q - 2 - i)
def Tight.y.{u_1} : (F : Type u_1) → [inst : Field F] → (q : ℕ) → {m : ℕ} → Fin m → Peel.C F q m :=
fun F [Field F] q {m} a => (Ideal.Quotient.mk (Peel.powIdeal F q m)) (MvPolynomial.X a)
@[reducible] def Peel.C.{u_1} : (F : Type u_1) → [Field F] → ℕ → ℕ → Type u_1 :=
fun F [Field F] q m => MvPolynomial (Fin m) F / Peel.powIdeal F q m
def Peel.powIdeal.{u_1} : (F : Type u_1) → [inst : Field F] → ℕ → (m : ℕ) → Ideal (MvPolynomial (Fin m) F) :=
fun F [Field F] q m => Ideal.span (Set.range fun i => MvPolynomial.X i ^ (q - 1))
def OddEquality.Mideal.{u_1} : (F : Type u_1) → [inst : Field F] → (q n : ℕ) → Ideal (Peel.C F q n) :=
fun F [Field F] q n => Ideal.span (Set.range fun P => ColOne.DPn F q P)
def ColOne.DPn.{u_1} : (F : Type u_1) →
  [inst : Field F] → (q : ℕ) → {n : ℕ} → ColComp.PerfMatch (Fin n) → Peel.C F q n :=
fun F [Field F] q {n} P => [] a with a < ↑P a, Tight.D F q a (↑P a)
def ColComp.PerfMatch.{u_1} : Type u_1 → Type u_1 :=
fun T => { P // ∀ (x : T), P x ≠ x ∧ P (P x) = x }
def OddEquality.DIdealZ : (q k : ℕ) → Ideal (FreeZ.CZ q (2 * k + 1)) :=
fun q k => Ideal.span (Set.range fun J => (Ideal.Quotient.mk (FreeZ.powIdealZ q (2 * k + 1)))
(OddEquality.DJZ q J))
def OddEquality.DJZ : ℕ → {k : ℕ} → BallotBound.Matching k → MvPolynomial (Fin (2 * k + 1)) ℤ :=
fun q {k} J => [] a with a ≠ 0 ∧ ↑J a ≠ 0 ∧ a < ↑J a, EveryField.DPR ℤ q (TheoremB.idx a) (TheoremB.idx (↑J a))
def OddEquality.MidealZ : (q n : ℕ) → Ideal (FreeZ.CZ q n) :=
fun q n => Ideal.span (Set.range fun P => (Ideal.Quotient.mk (FreeZ.powIdealZ q n)) (OddEquality.DPnZ q P))
def OddEquality.DPnZ : ℕ → {n : ℕ} → ColComp.PerfMatch (Fin n) → MvPolynomial (Fin n) ℤ :=
fun q {n} P => [] a with a < ↑P a, EveryField.DPR ℤ q a (↑P a)
def EveryField.DPR.{u_1} : (R : Type u_1) →
  [inst : CommRing R] → ℕ → {m : ℕ} → Fin m → Fin m → MvPolynomial (Fin m) R :=
fun R [CommRing R] q {m} a b =>
  ∑ i ∈ Finset.range (q - 1), (-1) ^ i * MvPolynomial.X a ^ i * MvPolynomial.X b ^ (q - 2 - i)
@[reducible] def FreeZ.CZ : ℕ → ℕ → Type :=
fun q m => MvPolynomial (Fin m) ℤ / FreeZ.powIdealZ q m
def FreeZ.powIdealZ : ℕ → (m : ℕ) → Ideal (MvPolynomial (Fin m) ℤ) :=
fun q m => Ideal.span (Set.range fun i => MvPolynomial.X i ^ (q - 1))

```

Paper (§1.5, §8)	Lean	Agreement
the box ring $C_{2k+1} = F[y_1, \dots, y_{2k+1}]/(y_i^r)$, $r = 2h + 1$	<code>Peel.C F q (2k+1) = MvPolynomial (Fin (2k+1)) F / powIdeal</code> , generated by $X_i^{(q-1)}$, at $q = 2h + 2$	$q - 1 = r$.
$D_r(a, b) = \sum_{u < r} (-1)^u a^u b^{r-1-u}$	<code>Tight.D F q a b = $\sum_{i < q-1} (-1)^i y_a^i y_b^{q-2-i}$</code>	The same sum, with $q - 1 = r$.
$D_{r,J}$: the product of $D_r(y_a, y_b)$ over the pairs of J that avoid 0; y_a is the coordinate of the index $a \geq 1$	<code>TheoremB.DJ F q J = $\prod_{\{a \neq 0, J a \neq 0, a < J a\}} \text{Tight.D F q (idx a) (idx (J a))}$</code> , $\text{idx } a = a - 1$	Literal: the coordinates of the box are numbered from 0, so the index $a \geq 1$ of the paper is $a - 1$ in Lean.
the ideal $(D_{r,J} : J \in \mathcal{J})$	<code>TheoremB.DIdeal F q k = span (range (D J F q))</code>	Literal.
$\mathcal{M} = \sum_P D_P \cdot C_N$ over the perfect matchings P of the $N = 2k + 2$ indices (Lemma 8.12)	<code>OddEquality.Mideal F q N = span (range (ColOne.DPn F q)), DPn F q P = $\prod_{\{a < P a\}} \text{Tight.D F q a (P a)}$</code> , <code>ColComp.PerfMatch</code> = fixed-point-free involutions	Literal.
the integral quotients of Corollary 8.13: $Z[y]/(y_i^r)$ modulo the ideal generated by the integer polynomials D_J , resp. D_P	<code>FreeZ.CZ q m / OddEquality.DIdealZ q k, FreeZ.CZ q N / OddEquality.MidealZ q N</code> , with <code>EveryField.DPR $\mathbb{Z}$</code> the same sum over $\mathbb{Z}$	Literal.

The proof of the upper bound in Lean (piece E15) uses only a perfect pairing of the box ring, the vanishing $(y_a + y_b) \cdot D_r(y_a, y_b) = 0$ in the box, and a degeneration to the r points $\{-h, \dots, h\}$ of $\mathbb{Q}$. Version 12 of the paper prints the same route (Lemma 8.12 and the proof of Corollary 8.13); version 11 used a Macaulay duality (Proposition 10.5(ii)) instead, and the piece E15 was written so as not to need it. Theorem O with equality is **not on the path** of Main Theorem', which uses only its inequality $\geq$ (`OddTheorem.theorem0`, piece E14). It is certified because the law of this project is that everything in our hands is proved.

2. What is not certified

- The topology.** Theorem 0(a), (c), (d) of the paper, i.e. [DS, Theorems 1.1(a) and 1.4], rest on Pham's theorem [Ph] and [DS, Theorem 2.2]. They identify the torsion of $R/(\psi_J)$ with that of $H_{2k}(X; \mathbb{Z})/L(X)$ and give the rank of $L(X)$. **Not formalized. Mathlib has no singular homology of complex Fermat varieties.**
- Hence the Main Theorem itself** — Conjecture 1.2 as a statement about homology — is certified only in its algebraic form, now for every degree.
- Not formalized, and not on the path of the proof:**
 - Corollary H (the integral Hodge conjecture) and Corollary W (partial Fermat varieties);
 - Theorem A of §10 and the counterexample of §11;
 - the verification record of §14;
 - the rest of §10 (Proposition 10.5, Corollaries 10.6 and 10.8), Proposition 2.6, the topological half of Proposition 2.1, and Lemma 12.4;
 - in Lemma 8.12, the injectivity of θ and the equality of its image (only the inclusion used is formalized);
 - [DS, Claim 4.3] for a subfamily $K \neq \mathcal{J}$ (only $K = \mathcal{J}$ is formalized);

- the graded refinement of Theorem B and of Theorem O (the dimension of each graded piece does not depend on F): optional in the pieces, not done.
- *Formalized although not needed*: Theorems 5.9 and 5.9', the $q = 3$ case of §4, Theorem O with equality (§1.4), Corollaries 7.8 and 9.12, and the case $r = 3$ of Theorem O without Pfaffians (Theorem 8.15).

4. **Two modules of numeric checks are not compiled on the author's Mac.** EvenAll.Checks (piece E20) and OddEquality.Checks (piece E15) evaluate counts such as $Q_k(m)$ at the paper's cells by decide. They contain no theorem used anywhere, and their kernel evaluation needs more than the memory cap of the local builds (4.5 GB); Aristotle reports that they compiled on its machines; this was not checked on the author's machine. The values they state are re-checked in CheckPares.lean at smaller cells, and the theorem modules they import are compiled locally.

5. **Faithfulness is an audit, not a theorem.** Lean guarantees that the proof is correct for the statement as written. That the Lean statement says what the paper says is checked in §1.3 by the author's audit. **A reader should check §1.2–§1.3 independently: about fifteen short definitions for Main Theorem' (§1.2), and seventeen more for Theorem O (§1.4).** This is the one place where trust in the author remains. The cold reader of §0 did this check, against the paper and against [DS] in the original, and found every definition faithful.

6. **Timing data and Aristotle's own notes.** Aristotle's own start and end times per task are shown on its web page but were not exported. The per-piece times in §6 are the download times of the result files. Aristotle's own account of its runs in the project, ARISTOTLE_SUMMARY.md, stops at piece E13; its notes on the later pieces, including the lists of «review suggested», were on its web page and are paraphrased in §5.

3. How the proof is organised in Lean

The odd degrees are the 29 pieces of version 1 (its §3), unchanged. The even degrees, and the odd box of Theorem O, were cut into **20 more pieces**, E1–E20, each one short self-contained Markdown file $q_*.md$ written from version 11 of the paper. The table gives each result its number in version 12, which keeps the numbering of version 11 except for rows E9 and E10 (see the note at the head of this certificate). Three passages of version 12 were rewritten to follow the routes taken in Lean: §8.5 (Lemmas 8.7' and 8.7, Lemma 8.8, Corollary 8.9: the route of pieces E9 and E10, with no exterior algebra), the upper bound in the proof of Corollary 8.13 (the route of piece E15), and the proof of Lemma 9.10 in the case $0 \notin \mathcal{C}_1$ (the route of piece E19, without the injectivity of the map θ). Piece E15 (the equality in Theorem O) was sent last, by the order fixed in advance: first everything on the path of Main Theorem', then the rest.

Each piece produced one new folder under RequestProject/. **No piece ever modified a file of an earlier piece:** every returned project was compared file by file with the compiled project (*diff -rq*), and every earlier .lean file was byte-identical.

#	Source file	Paper v12	Lean folder	Main Lean results
E1	q_any_bip.md	§7 for every box $q \geq 2$ (Theorem 7.6, Theorem C; see the note below the table)	BipAny	thm76_any, theoremC_any, choose_pred_prime_pow, thm76_prime_pow, theoremC_prime_pow

#	Source file	Paper v12	Lean folder	Main Lean results
E2	q_even_count.md	Lemma 9.1; Theorem 0(b) for even m and the full family $K = \mathcal{J}$	EvenCount	QkEven, QkEven_eq, card_closedPointed, card_Gamma_even, finrank_IK_le_even
E3	q_odd3.md	Theorem 8.15 ($r = 3$, no Pfaffians)	Odd3	0_ge_three, I_odd_one_eq_DIdeal, card_Z_le_finrank
E4	q_pow2_leading.md	Proposition 9.2 (degree 2^n , leading forms)	Pow2	prop92_ge, quartic_char_two, pow2_of_oddbox, lowest_forms
E5	q_quartic_assembly.md	§9.5, Theorem 9.11 at $m = 4$; the rest of its proof under a hypothesis $H(m, k)$ (see the note below the table)	EvenAssembly	HypH, theorem_i/ii/iii, mainTheorem'_four
E6	q_oddbox_shapes.md	§8.1–§8.2: identities, shapes, Lemma 8.3	OddShapes	(8.1)–(8.4): A0, A1, A2_D, A2_Dminus, A3_D, A3_Dminus, A4_D, A4_Dminus, A4_coeff_D, A4_coeff_Dminus; optS_filter_eq_Icc, isInterlaced_root_even/odd
E7	q_oddbox_layers.md	Lemmas 8.4, 8.5	OddLayers	isInterlaced_layerS, card_ZS_root_even/odd
E8	q_pfaffian.md	§8.3, bordered Pfaffians	Pfaffian	pf_eq_sum_matchings, bpf_laplace, bpf_expand_var, (C1)–(C6)
E9	q_rank_two.md	§8.5: Lemma 8.8 (border shift, rank-two update), Corollary 8.9	RankTwo	R2, R4, R4' (Lemma 8.8(ii) in the form printed in the paper), S2, S5a, S5b
E10	q_membership.md	Lemmas 8.7' and 8.7	Membership	M2a, M2b, M3_pos, M3_zero, V1
E11	q_oddbox_patterns.md	Definition 8.6, the ideals V_Λ	OddPatterns	mPf, vS, vSAll, C3, D3
E12	q_oddbox_lifts_1.md	Prop. 8.10, first cases	OddLifts	P1–P3, T1–T5
E13	q_oddbox_lifts_2.md	Prop. 8.10, the cases that change the mark	OddLifts2	Q1, Q2, Q'1, Q'2, T6–T8
E14	q_oddbox_theorem.md	Prop. 8.10 (its case analysis is the piece's Lemma K; see the note below the table), Theorem 8.11, Theorem $O \geq$ (§8.7)	OddTheorem	lemmaK, prop810, theorem811, theorem0
E16	q_even_blocks.md	Lemmas 9.4, 9.5	EvenBlocks	lemma94, exists_setting, SInv, lemma95
E17	q_even_count_colours.md	Lemma 9.6	EvenColours	C2, C4_even, C4_odd
E18	q_even_block_minus.md	Lemmas 9.7–9.9	EvenMinus	lemma97, lemma98, lemma99, lemmaM5, lemmaM6

#	Source file	Paper v12	Lean folder	Main Lean results
E19	q_even_block_one.md	Lemma 9.10; Theorem 9.11 $\geq$	EvenOne	lemma910, E1–E3
E20	q_even_assembly.md	Theorem 9.11; Main Theorem' for every m ; Corollaries 9.12, 7.8	EvenAll	A3, B1, B2, mainTheorem', D2, E1, E2
E15	q_oddbox_equality.md	Lemma 8.12 in the direction used (A5: $\theta(\mathcal{M}) \supseteq (D_p)C$; the injectivity of θ is not formalized), Corollary 8.13: Theorem O with equality, freeness	OddEquality	D1, D2_DJ, D2_M, A5, B3, partC

Note to the table. The third column names results of the paper. Some pieces gave their own names to intermediate statements, and the Lean files keep them: in E1, «Theorem 7.6'» and «Theorem C'» are Theorem 7.6 and Theorem C of the paper, which version 12 states for every box $q \geq 2$ over a field in which the binomial coefficients $C(q-1, t)$ are non-zero (§1.9), and «Lemma P», «Corollary P» say that these coefficients are non-zero in characteristic p when $q = p^v$; in E14, «Lemma K» is the case analysis inside the proof of Proposition 8.10 (which of the constructions of that proof applies to a given tail); in E5, « $H(m, k)$ » is the hypothesis `EvenAssembly.HypH m k` of step 4 below, proved for every even m in E20.

3.1 The chain of the final theorem, even m

Every arrow is a Lean proof.

- Upper bound, every field** (E2, `finrank_IK_le_even`): $\dim_F I_F \leq Q_k(m)$, by the route of version 1 — $\dim_F I_F = \text{rank}_F(A)$ for the integer matrix A ; $\text{rank}_F \leq \text{rank}_\mathbb{Q}$; over $\mathbb{C}$, evaluation at the m -th roots of unity, now with the root -1 , gives $|\Gamma|$; and $|\Gamma| = Q_k(m)$ (Lemma 9.1). This is [DS, Claim 4.3] for even m , re-proved without topology.
- Lower bound at a prime $p \mid m$, every cofactor** (E19, `EvenOne.E3`), over the algebraic closure of F_p , with $m = p^v \cdot r'$, $p \nmid r'$:
 - $\dim I = \sum_c \dim \pi_c(I)$ over the colourings c (Lemma 9.4 = Lemma 6.1 with no parity, E16);
 - each term is a product of block dimensions (Lemma 9.5, E16), now with the self-inverse colours 1 and -1 ;
 - the count factors in the same way, with sum $Q_k(m)$ (Lemma 9.6, E17);
 - each block dimension is at least its count: the pair blocks by Theorem C for every box $q = p^v$, the prime 2 included (Lemma 9.7, E18 with E1); the colour 1 at an odd prime by Theorem B (Lemma 9.8, E18); the colour -1 at an odd prime by Theorem O at the box p^v (Lemma 9.9, E18 with E14); the colour 1 at the prime 2 by Proposition 9.2 and Theorem O at the box $2^v - 1$ (Lemma 9.10, E19 with E4 and E14).
- Theorem O, $\geq$** (E14, `OddTheorem.theorem0`), from the Pfaffians and patterns of E6–E13.
- $H(m, k)$ for every even m** (E20, A3). $H(m, k)$ is `EvenAssembly.HypH m k`: for every prime $p \mid m$, the rank over F_p of the integer matrix A is $Q_k(m)$. Steps 1 and 2 give $\text{rank}(A) = Q_k(m)$ over the algebraic closure of F_p , and the rank does not change from F_p to its algebraic closure (`rank_map_algebraMap`, version 1).
- Every field and freeness over $\mathbb{Z}$** (E5 with E20: `EvenAssembly.theorem_ii`, `theorem_iii`; `EvenAll.B1`, `B2`): $\text{rank}_F(A)$ depends only on the prime field; characteristic 0 and $p \nmid m$ from step 1 and the count of Γ ; $p \mid m$ from $H(m, k)$; freeness by the integer-matrix lemma of version 1 (piece 14).

6. Every $m \geq 1$ (E20, `mainTheorem'`): odd m by version 1, even m by step 5.

3.2 The dependency cone

A short Lean program (`checks/DepsPares.lean`, `log CERTIFICADO_PARES/deps_pares_postclean.log`, identical line for line to the run before the clean rebuild) collects every declaration that a theorem uses, directly or indirectly, and keeps those defined in the project.

Theorem	Project declarations in its cone	Project modules in its cone
<code>EvenAll.mainTheorem'</code> (the final theorem)	4 412	180
<code>EvenAll.B2</code> (Theorem 9.11, even m)	4 169	174
<code>OddEquality.D2_DJ</code> (Theorem O, freeness)	2 470	101
<code>OddEquality.D1</code> (Theorem O with equality)	2 425	97

The counts include the auxiliary declarations Lean generates (equation lemmas, *match* functions, proof terms). The project has 233 modules; 53 are outside the cone of the final theorem. Among them are the whole folder `OddEquality` (piece E15: Theorem O with equality is not on the path, as said in §1.4), `Odd3` (piece E3: the case $r = 3$ is covered by the general Theorem O), the modules of numeric checks, and the summary modules *Main* of several folders, which only import and document. The full list is in `CERTIFICADO_PARES/deps_pares_postclean.log`.

4. Trust base

- **What must be trusted:**
 - the Lean 4 kernel, toolchain `leanprover/lean4:v4.28.0`;
 - Mathlib at commit `8f9d9cff6bd728b17a24e163c9402775d9e6a365` (tag `v4.28.0`, `lake-manifest.json`);
 - the three standard axioms `propext`, `Classical.choice`, `Quot.sound`;
 - the audit of the statement (§1.3).
- **What need not be trusted:** Aristotle (Harmonic), the AI system that wrote the Lean proofs, and the author's brute-force checks. Every theorem module Aristotle returned was compiled again on the author's machine, and the kernel re-checks every proof; the two modules of numeric checks, and the two *Main* files that only import them, exceed the memory cap and were not (§2 item 4).
- **Forbidden constructs**, searched in all 233 files: none. No sorry, no admit (the string occurs only in the English words «admits», «admitting» inside two comments), no axiom, no opaque, no unsafe, no partial def, no `@[extern]`, no `implemented_by`. `native_decide` occurs only in three comments, each saying that it is **not** used (`EvenAll/Checks.lean`, `EvenOne/Checks.lean`, `OddEquality/Checks.lean`).
- **The tactic decide** occurs in 12 files (3 of them from version 1): small finite facts such as $Q_k(m)$ at given cells, parities and inequalities of naturals. It is kernel-checked and adds no axiom, as `#print axioms` confirms. The word *decide* also occurs, as the Boolean function *decide* (p), in a lemma name or in comments, in 10 more files.
- **Options.** The files set resource limits (`maxHeartbeats`, `synthInstance.maxHeartbeats`, `maxRecDepth`, `synthInstance.maxSize`), printing options (`pp.*`) and elaboration options (`autoImplicit false`, `relaxedAutoImplicit false`). None affects soundness.

- **Instances.** The project declares 16 instances, 13 of them already in version 1. They are structures Mathlib already provides (a ring as a module over itself, `Semiring.toModule`; the ring and algebra structures of a quotient), and facts: finiteness of a module or a type, decidability of a predicate, `NeZero`, `Nontrivial`. The three new ones are `Pfaffian` (decidability of «is a matching»), `EvenColours.qNeZero` and `EvenOne.instModuleGASelf` (`Semiring.toModule`, declared with high priority to shorten instance search). None introduces a new structure, and instances add no axiom.
- **Axioms**, printed after each piece (logs `check_runE*.log`) and again in `check_pares_postclean.log`: always `[propext, Classical.choice, Quot.sound]` or a subset of it. Over the twenty final check logs, 413 lines of `#print axioms`: 401 give the three axioms, 11 give `[propext, Quot.sound]` and 1 gives `[propext]` (small combinatorial lemmas about lengths, ranges and parities in `OddPatterns`, `OddLifts`, `OddLifts2`, `OddTheorem`, `Pfaffian`, `RankTwo`). No line names any other axiom, and none contains `sorryAx`. In `check_pares_postclean.log` the eleven main theorems give the three axioms.
- **The compiled files, through the kernel again.** The cold reader of §0 re-submitted to the Lean kernel, from the compiled `.olean` files, every project declaration on which `EvenAll.mainTheorem'`, `OddEquality.D1`, `D2_DJ`, `D2_M` and `OddTheorem.theorem0` depend: 4 669 declarations, of which 4 650 theorems and definitions were re-checked (type and proof term) and the other 19 are 9 inductive blocks, re-submitted and accepted, with the generated recursor equal to the compiled one; a negative control (a wrong proof) was rejected. So no compiled file on the path holds a declaration that did not pass the kernel. The author repeated both runs with the reader's files on 4 October 2026 and obtained the same output (`projecto_lean/sample_L5_kernelcheck.log`, 152 s; `projecto_lean/sample_L6_inductives.log`, 110 s). Mathlib's own compiled files are not re-checked by these runs.
- **Aristotle's own register** for the project lists 359 properties PROVED, 0 NEGATED, 0 in progress.

5. The procedure

The procedure of version 1 (§5 there), applied to each of the 20 pieces, with four additions learned on the way:

1. **The whole cycle in one turn.** When a piece came back: download, diff, grep, build, check file, axioms, faithfulness audit, register; then write the next piece, brute-force it, send it, register the send.
2. **One module at a time under a watchdog** (`vigia.sh`): every local lake build of a module ran alone, with a cap of 4.5 GB of real memory and 900 s; the cap was never raised.
3. **A check file per piece** (`checks/CheckE<N>.lean`) restates the main statements **by hand**, with every type written out, and Lean must accept the restatement against the proved theorem. A restatement that does not typecheck stops the cycle.
4. **Controls that can fail.** Every brute-force check before sending carries negative controls (a deliberately wrong variant); a control that cannot fire (for instance one that negates a statement true in every cell tested) was replaced, and the silent ones are declared in the logs.

Stop conditions, fixed in advance: a property NEGATED by Aristotle; a statement that is not faithful; a build failure; a sorry or an extra axiom; a change to an old file. **None occurred**, apart from the two modules of numeric checks that exceeded the memory cap of the local build (§2 item 4); they contain no theorem used elsewhere.

«**Review suggested**». Aristotle's page showed this badge on several pieces (among them E15, E18, E19, E20). Every time it accompanied Aristotle's own list of formalization choices: hypotheses that were dropped

(statements more general than asked), a different but equivalent form, an added hypothesis stated openly, or parts of a piece weaker than written but sufficient for the argument. Each was read and judged in the audit log. In E15 the choices were: (C3) needs T nonempty (true in the use, $0 \in T$); (C1) stated for an arbitrary finite point set; (C5) proved as a surjection, which is all the argument needs; the optional graded statement (D3) not done.

6. Data per piece

- **Tarball, md5:** the project returned by Aristotle (project d1c5d0ac-9ffc-477b-ae34-0729a44d985c), file in ~/Downloads, first 8 hex digits of its md5.
- **Downloaded:** the local time the file was saved (CEST).
- **Build:** the local lake build of the new folder, one module at a time; seconds summed over the modules, and the largest real-memory peak measured by the watchdog. Each module's time includes Lake's replay of the ~8 000 earlier jobs (about a minute).
- **Files / lines / theorems / definitions:** of the folder, in the final project.
- **Checks:** the brute-force check made before sending.

Table 6a. Provenance (the source file of each piece is in the table of §3).

Piece	Task	Tarball	md5	Downloaded	Brute force before sending
E1	423d8b51	(32)	a81a346e	02 Oct 13:27	51 down-sets of 51, two programs (note d)
E2	d5d6af65	(33)	3551684a	02 Oct 14:13	64 checks, 0 failures
E3	c5108d14	(34)	9f16d524	02 Oct 15:27	195 checks, 0 failures
E4	1955630a	(35)	34928d5b	02 Oct 16:11	106 checks, 0 failures
E5	ca046a54	(36)	8a8ac7d1	02 Oct 17:36	21 checks, 0 failures
E6	2c5d9af9	(37)	c714a335	02 Oct 18:26	93 checks, 0 failures
E7	bc21e06d	(38)	f398fdaa	02 Oct 19:39	120 checks, 0 failures
E8	c5d2ce6f	(39)	42b8e03b	02 Oct 20:59	4491 checks, 0 failures
E9	225935e3	(40)	e73b6a2f	02 Oct 22:16	1718 checks, 0 failures
E10	e52c7421	(41)	b604b427	02 Oct 23:06	840 checks, 0 failures
E11	8837ccde	(42)	ee6077bc	02 Oct 23:55	1170 checks, 0 failures
E12	a6ec0ca7	(43)	17cbd837	03 Oct 07:23	8261 checks, 0 failures
E13	e8d51073	(44)	56d29b7f	03 Oct 08:40	2269 checks, 0 failures
E14	3cbb9511	(45)	ba6c0325	03 Oct 09:12	8743 checks, 0 failures
E16	deaa828d	(46)	658746a6	03 Oct 10:20	3399 checks, 0 failures
E17	f708fa43	(47)	cfdae57a	03 Oct 11:34	1869 checks, 0 failures
E18	020121f8	(48)	2e787469	03 Oct 12:23	84 checks, 0 failures
E19	5961f1c2	(49)	c179c0bb	03 Oct 13:30	118 checks, 0 failures
E20	fa752fdd	(50)	ba6d6466	03 Oct 15:38	59 checks, 0 failures

Piece	Task	Tarball	md5	Downloaded	Brute force before sending
E15	214cca93	(51)	eac51a96	03 Oct 16:51	111 checks, 0 failures

Table 6b. The Lean folder and its local build.

Piece	Folder	Files	Lines	Thms	Defs	Modules built	Build s	Peak GiB	Killed attempts
E1	BipAny	6	633	28	0	6	411	1.87	two (notes a, b)
E2	EvenCount	5	825	24	5	5	346	1.76	—
E3	Odd3	6	1150	74	15	6	397	1.83	—
E4	Pow2	4	978	50	12	4	255	2.16	—
E5	EvenAssembly	3	253	14	1	3	172	2.07	—
E6	OddShapes	6	1056	57	13	6	397	1.80	—
E7	OddLayers	6	850	48	5	6	481	1.98	—
E8	Pfaffian	9	2132	105	22	9	589	1.77	—
E9	RankTwo	8	1152	54	9	8	463	2.07	—
E10	Membership	5	552	23	6	5	342	1.93	—
E11	OddPatterns	7	770	38	6	7	495	2.13	—
E12	OddLifts	4	1099	40	2	4	391	1.83	—
E13	OddLifts2	5	1107	36	2	5	429	1.70	—
E14	OddTheorem	5	504	14	1	5	335	1.87	—
E16	EvenBlocks	6	929	43	13	6	410	1.98	—
E17	EvenColours	7	1454	51	14	7	483	1.89	—
E18	EvenMinus	7	1501	66	13	7	440	1.99	—
E19	EvenOne	6	957	48	11	6	448	2.25	—
E20	EvenAll	6	869	42	2	4	296	1.85	Checks (note c)
E15	OddEquality	9	1346	67	26	7	629	1.74	Checks (note c)

Notes. (a) The first local build of BipAny was started under the general cap of 1.2 GB (the wrong cap for a Lean build) and was killed; (b) a second attempt built modules in parallel and was killed; the build that counts ran one module at a time under the Lean cap and finished. (c) The module of numeric decide checks exceeded the Lean cap of 4.5 GiB and was killed; its *Main* (which only imports and documents) was therefore not built (§2 item 4). (d) Recorded in `q_any_bip.md`, item 2. Build seconds are summed over the modules that finished; the peak is the largest real-memory peak measured by the watchdog, in GiB (kB/1 048 576), the unit of the 4.5 GiB cap.

7. Totals

	Files	Lines	Theorems and lemmas	Definitions
Version 1 (odd degrees, 29 pieces)	113	21 126	937	—
This version, pieces E1–E20	120	20 117	922	178
Whole project	233	41 243	1 859	—

- **sorry: 0.** Theorems and lemmas are counted as lines beginning with *theorem* or *lemma* ; definitions (pieces E1–E20 only) as lines beginning with *def*, *noncomputable def* or *abbrev* (so *structure*, *instance* and *noncomputable abbrev* are not counted). The project counts were recomputed after the last piece, not added up.
- **Local build of the twenty new folders:** 8 209 s of module builds (about 2 h 17 min), never above 2.25 GiB of real memory.
- **Brute-force checks before sending:** 33 731 checks over E2–E20, 0 failures, plus the 51 down-sets of E1; every check file also carried negative controls that fired.
- **Calendar:** piece E1 sent on 2 October 2026 at about 13:00 CEST; piece E15 downloaded on 3 October at 16:51 CEST: about 28 hours for the twenty pieces.
- **Aristotle's register** for the project, read on its page on 3 October 2026 at about 17:30: 359 properties PROVED, 0 NEGATED, 0 in progress.

8. Reproducibility

Machine: Apple Mac (arm64), 8 CPU cores, 8 GB of memory, macOS (Darwin 25.5.0).

Toolchain: `leanprover/lean4:v4.28.0;` `Mathlib` `v4.28.0` `=` `commit 8f9d9cff6bd728b17a24e163c9402775d9e6a365.`

Commands, from the project root (`output-final_aristotle/`):

```
lake exe cache get                # Mathlib's compiled files
lake build RequestProject.EvenAll.PartBC RequestProject.EvenAll.PartD RequestProject.EvenAll.PartE
RequestProject.OddEquality.PartD RequestProject.ColAssembly.Main
lake env lean ../../checks/CheckPares.lean
lake env lean ../../checks/CheckTheorem0.lean # the definitions behind Theorem 0 (§1.4)
```

In the published repository [Rep] (and in the Zenodo archive, which holds the same folder), the project root is `lean/project/`, the folder `CERTIFICADO_PARES/` is `lean/even-degrees/logs/`, the check files `checks/` are `lean/even-degrees/checks/`, and the per-piece logs of `proyecto_lean/` are in `lean/even-degrees/logs/` too. From `lean/project/` the last two commands are `lake env lean ../even-degrees/checks/CheckPares.lean` and `lake env lean ../even-degrees/checks/CheckTheorem0.lean`. The kernel re-check of §4 is in the cold reader's `record/cold-readings/17_internal-reader_on-lean-certificate-v2/checks/L5_kernelcheck.lean` and `L6_inductives_finite.lean`, run the same way.

The second command builds the five modules that `CheckPares.lean` imports, and everything they import: 216 of the 233 modules. The final theorem `EvenAll.mainTheorem'` lives in `EvenAll.PartBC`. On a machine with little memory, build the modules one at a time, as the author did.

Clean rebuild. On 3 October 2026 the project's own compiled files (`.lake/build`) were moved aside, unchanged, to `.lake/build_pre_clean_2026-10-03`; Mathlib's compiled files (`.lake/packages`) were kept. Then

every module of the project was compiled again from its source, one at a time, each with its own `lake build` under the watchdog with the cap of §5, in an order where every module comes after all the modules it imports (CERTIFICADO_PARES/orden_modulos.txt). The script and its logs are in CERTIFICADO_PARES/: `clean_rebuild_pares.sh`, `clean_rebuild_2026-10-03.log`, and one log per module, `cr_logs/<module>.log`. The result:

- **229 of the 233 modules were built, all with exit code 0**, from 17:20:22 to 21:00:12 CEST (3 h 40 min). The module builds sum to 13 190 s; the longest is `Peel.Main`, 93 s.
- The largest real-memory peak was **2.35 GiB** (`EveryField.Main`), about half the cap of 4.5 GiB.
- **Four modules were not built, as declared in §2 item 4 and note (c) of §6**: `EvenAll.Checks` and `OddEquality.Checks` (numeric decide checks above the cap), and `EvenAll.Main` and `OddEquality.Main`, which only import them and document. No other module imports any of the four.
- Every one of the 229 compiled `.olean` files is newer than the start of the rebuild.
- The sources did not change: after the rebuild, the md5 of each of the 233 `.lean` files is the one in the manifest below.

Then `CheckPares.lean` and `DepsPares.lean` were run again against the rebuilt files: logs `check_pares_postclean.log` (210 s, peak 1.35 GiB) and `deps_pares_postclean.log` (188 s, peak 1.49 GiB), both with exit code 0. **Their output is identical, line for line, to the runs before the rebuild** (`check_pares.log`, `deps_pares.log`), except the two lines of `#print ColUpper.tU`, added between the two runs (§1.2). Neither log contains *error*, *warning* or *sorry*. The two final logs quoted in §1 and §3.2 are from the rebuilt files; the per-piece logs `check_runE*.log` cited in §1.4 and §4 are from the builds of §6.

Fingerprints of the source: the md5 of every one of the 233 `.lean` files is listed in CERTIFICADO_PARES/MANIFEST_md5_233.txt (paths sorted bytewise); the md5 of that list is `8ac0df81107480be80e53c75b2e4dbbd`. `lakefile.toml` `d3ae68ec...`, `lean-toolchain` `b8b2923c...`, `lake-manifest.json` `235e6523...`.

9. Changes from version 1

- **Scope:** Main Theorem' for every degree $m \geq 1$ (version 1: odd m), and Theorem O with equality and freeness.
- **20 new pieces**, E1–E20; the 29 pieces of version 1 and their files are unchanged.
- **New final theorem:** `EvenAll.mainTheorem'`; `ColAssembly.mainTheorem'` is kept as the odd case.
- **Procedure:** the four additions of §5.

Corrections after the cold reading (4 October 2026; §0). The reader's findings, each checked by the author before it was applied: the numbering of §8.5 against version 11 (line 8, §3); three rewritten passages, not two (§3); 16 instances, 13 of them from version 1 (§4); 178 definitions by the stated rule, not 187, and eight rows of Table 6b (§6, §7); the tactic `decide` in 12 files, not 22 (§4); Lemma 8.12 in one direction, Theorem O(b) for $K = \mathcal{J}, R4'$ and the labels A0–A4 in the table of §3, and a longer list of what is not formalized (§2); the seventeen definitions behind Theorem O, now printed and compared (§1.4, new check file `CheckTheorem0.lean`); two sentences on the four modules not built locally (§4, §5); and, in the pdf, no more hyphens inside code. Added: the cold reading itself (§0), the kernel re-check of the compiled files (§4), and the commands in the layout of the published repository (§8). No theorem, statement or number of a Lean result changed.

10. What this certificate claims, in one paragraph

A computer proof assistant, Lean 4 with the Mathlib library, has checked a complete proof of the following statement, with no gap and no axiom beyond the three standard ones:

For every $m \geq 1$ and every $k \geq 0$, the quotient of the integral group ring $Z[(Z/m)^{2k+1}]$ by the ideal generated by the Degtyarev–Shimada elements ψ_j is a free abelian group of rank $m^{2k+1} - Q_k(m)$, and over every field the corresponding quotient has that dimension.

By [DS, Theorem 1.1(a)] (Pham's theorem and the intersection numbers of [DS, Theorem 2.2], cited and not formalized), this is equivalent to Conjecture 1.2 of Degtyarev and Shimada for the Fermat varieties of degree m in even dimension $2k$, for every $m \geq 3$.

The proofs in Lean were written by the AI system Aristotle (Harmonic) from pieces written from the paper. The audit, by the author assisted by Claude (Anthropic), checked that the statements match the paper, and compiled and re-checked everything on the author's machine. Before release, a separate reader read this certificate cold, compared the statements with the paper and with [DS] in the original, and re-checked the compiled proofs through the Lean kernel; its verdict was «holds as far as I can check».

References

- [DS] A. Degtyarev, I. Shimada, *On the topology of projective subspaces in complex Fermat varieties*, J. Math. Soc. Japan **68** (2016), no. 3, 975–996; doi:10.2969/jmsj/06830975; arXiv:1405.4683v3.
- [Ph] F. Pham, *Formules de Picard–Lefschetz généralisées et ramification des intégrales*, Bull. Soc. Math. France **93** (1965), 333–367.
- R. Amichis Luengo, *The Chaise Longue Theorem*, Zenodo, concept DOI 10.5281/zenodo.22961150.
- R. Amichis Luengo, *The Chaise Longue Theorem — Lean certificate*, v1, Zenodo, DOI 10.5281/zenodo.23045371; v2 (this certificate and its project), DOI 10.5281/zenodo.23129677; all versions, concept DOI 10.5281/zenodo.23045370.
- [Rep] R. Amichis Luengo, *The Chaise Longue Theorem — working record*, <https://github.com/tretoef-estrella/chaise-longue-theorem> (the Lean folder is `lean/`).
- The Lean 4 theorem prover, <https://lean-lang.org>; the Mathlib library, <https://github.com/leanprover-community/mathlib4>.
- Aristotle, Harmonic, <https://aristotle.harmonic.fun>.